Day 3

The Swisscom Shape & Production Readiness

From one MSR to one Swisscom environment, then make it operable

Block 5 · The Swisscom shape

Block 6 · Production readiness & operations

Day 3: Agenda

Block 5 · 09:00–13:00

- 400+ packages: build-time vs. start-time, wpm
- webMethods image build alternatives
- Multi-stage MSR build, layer optimisation
- GitLab CI/CD: build, scan, push
- Swisscom architecture: clustering, load balancing
- API Gateway, TN/AT, UM, JDBC Adapter
- 🏆 Capstone 5: 2 × MSR, nginx, API Gateway

Block 6 · 13:00–17:00

- Externalized config & secrets, Ansible, 4 environments
- 🏆 Capstone 6: secrets + prod overlay
- Graceful shutdown, rolling updates
- Graylog and Nagios/OP5
- Dos & Don'ts, security, migration
- HA and the Kubernetes outlook
- Final discussion, next steps

Hands-on: *Exercises 5.1–5.3 and 6.1–6.3 · Capstone 5 (70 min) · Capstone 6 (≈ 90 min)*

Classification - Internal



Block 5

The Swisscom Shape

400+ packages · Build alternatives · CI/CD · Clustering · API Gateway

The Challenge: 400+ Packages in a Container World

- Classic model: install once on a long-lived server, deploy packages incrementally for years
- Containers invert this: the image is immutable and disposable, so every package change means a new image
- MSR also changes the unit of deployment: an image can carry all packages of one IS today, or 1–n packages per microservice tomorrow
- Three ways to get packages into a container:
  - Bake at build time: COPY or wpm install in the Containerfile; reproducible, the recommended default
  - Install at start: wpm in an entrypoint; small image, but start-up depends on a registry
  - Externalize: EXTERNALIZE_PACKAGES + volume; closest to today, loses immutability

Gotcha: Measure start-up time with your real package set early: it drives the split between images.

Classification - Internal

Build-Time vs. Start-Time Package Deployment

Bake in at build time

- The image IS the release: identical in Dev, Test, Prod
- No dependency on a package registry at start
- Larger images, longer builds
- Best for: everything that is released together
- A change means rebuild + rolling update

Install at container start

- Small base image, quick builds
- Needs registry access and tokens at EVERY start
- Start-up time and failure modes depend on an external system
- Best for: Dev loops, sandboxes
- Two containers from one image may differ: hard to audit

Recommendation for Swisscom: bake packages into versioned images grouped by release unit; use bind-mounted packages only on developer laptops (Capstone 2 Part 4).

Classification - Internal

wpm: the webMethods Package Manager in the Image

Containerfile (excerpt)

```
FROM ibmwebmethods.azurecr.io/webmethods-microservicesruntime:12.1.0.3
# wpm ships in the image: /opt/webmethods/wpm/bin/wpm

# IBM packages from the webMethods package registry (token from IBM)
RUN --mount=type=secret,id=wpm_token \
    wpm install -ws https://packages.webmethods.io -wr licensed \
        -j "$(cat /run/secrets/wpm_token)" WmJDBCAdapter

# Swisscom packages straight from a Git repository
RUN --mount=type=secret,id=git_token \
    wpm install -r https://gitlab.swisscom.example/wm/packages.git \
        -u ci -p "$(cat /run/secrets/git_token)" OrderManagement
```

Build secrets (--mount=type=secret) keep tokens out of image layers. The Git URL is a placeholder for Swisscom's GitLab.

Classification - Internal



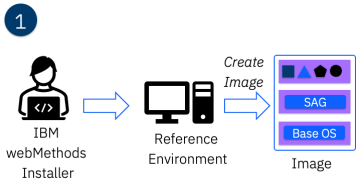
Building Custom webMethods Base Images

Options

IBM webMethods Container Image Build Alternatives

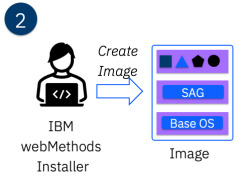
Build from Reference Environment.

- Install SAG components via Installer
- Use shipped Dockerfiles



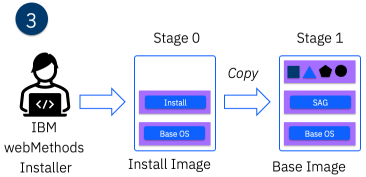
IBM Installer image creator

- Linux CLI SAG installer feature
- Uses DIND (Docker in Docker)



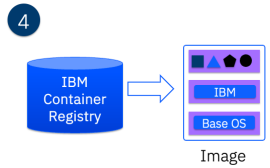
Multistage Dockerfile

- Create Reference Installation in Docker
- Use shipped Dockerfile in final stage



Container Registry

- No build process for base images required
- Provided “As is” (no custom layered products)



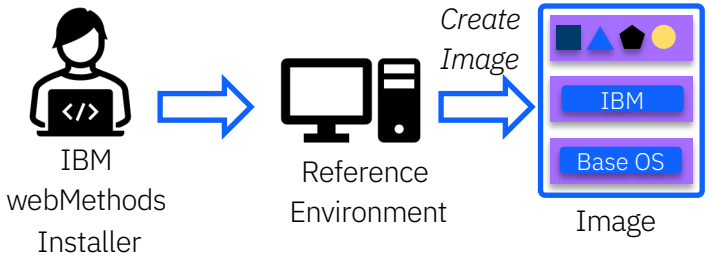
Classification - Internal

webMethods Container Image Build: Reference Environment

Outdated

Expert
abs

1



Build Process:

1. Use IBM webMethods Installer / CCE to install webMethods components
2. Create Dockerfiles via shipped docker scripts (../docker directory)
3. Create docker image via docker build
4. Push Image to your registry

+ Can use UpdateManager to apply engineering fixes or component fixes which are not yet released in base images of the IBM webMethods Container Registry
- Unpredictable results – reference environment could be changed

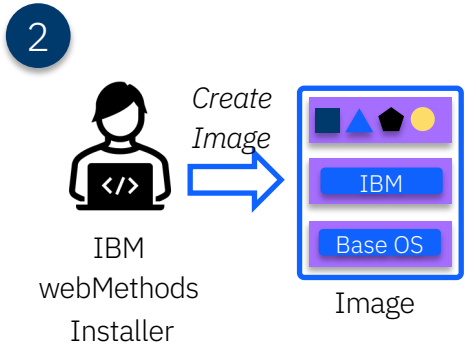
→ No longer recommended

Classification - Internal

webMethods Container Image Build: Installer Image Creation

Expert
Labs

Deprecated



Build Process:

1. Use IBM webMethods Installer / CCE to create webMethods Container images

2. Push Image to your registry

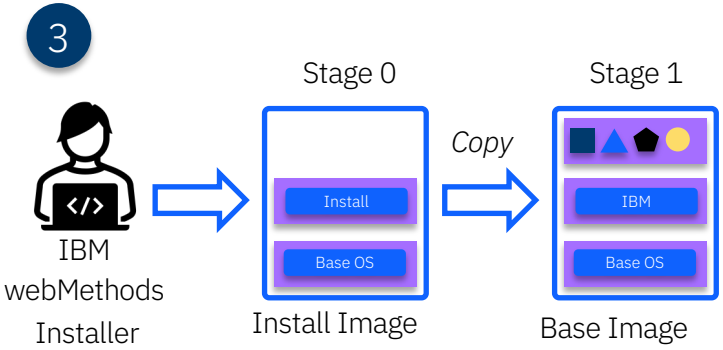
+ Easy and fast way to produce custom images with custom layered components
+ Built-In component list documentation
+ Applies latest fixes by default
+ Specific official fixes can be selected if required
- Uses DIND (Docker in Docker) and requires privileged access to docker daemon
- Works only on Linux systems
- Not available for all components
- Deprecated already for some components (API Gateway)

→ Applicable only for custom build scenarios, where privileged access to Docker daemon is allowed (e.g. dedicated build VMs)
→ Deprecated – will be removed in future versions

Classification - Internal

webMethods Container Image Build: Multistage Dockerfile

Recommended



Build Process:

1. Use SAG Installer inside Stage 0 for a reference installation (replay pre-recorded installation script)

2. Stage 1 of Docker build process will copy necessary files from Stage 0 reference and adjust configs accordingly

\+ Standard Build Process as for On-Prem Environments
\+ Allows using also UpdateManager for applying engineering fixes
\+ Runs also in non-priviledged environments (Kubernetes, Openshift)
\- IBM webMethods Installer script needs to be recorded first
\- Dockerfiles must be created manually (extracted from Stage 0)
\- Applying fixes requires either re-recording or use always latest fixes

\- → Recommended for custom build scenarios, where privileged access to Docker daemon is NOT allowed (e.g. dedicated build VMs) and Hotfixing is a critical requirement

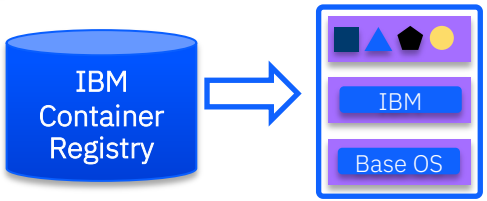
Classification - Internal

webMethods Container Image Build: Multistage Dockerfile

Recommended

Expert
abs

4



Image

123

Build Process:
- none / derive from base image if required -

+ No custom build process required
+ Custom Fixes can be supplied through Dockerfiles from support

→ Recommended for platform images such as Universal Messaging, Terracotta Big Memory, API Gateway, Developer Portal
→ Might not be applicable for MSR requiring Adapters or layered Products
→ Required for MWS, TN, BPMS, TaskEngine

Classification - Internal

Comparison webMethods Container Image Build Alternatives

	Reference Environment	Installer Image Creation	Multistage Dockerfile	IBM Container Images
Restrictions	None	Not available for all products.	None	Not available for solution images, Layered products / components currently not available (e.g. specific Adapters, Process Engine, etc)
Benefits	No restrictions – Ref environment can be changed as required.	Easy creation process – built-in documentation for available components.	Cloud-native way, should be applicable for ALL build scenarios.	Zero Build Requirements
Downsides	Requires Installation / VM or automated way of installing products. Ref environment can be changed as required.	Uses DIND. Limited to privileged environments with access to Docker daemon.	Dockerfile must be extracted from an install-base image first.. More complex than wM Installer image builds.	Hot Fixes cannot be applied. Requires custom builds from Support if required.
Recommended	-	Applicable for most MSR build scenarios where a Docker daemon can be used (mainly on VMs)	Recommended for non-privileged environments / Kubernetes / OpenShift build nodes. Air-gapped environments and special Base OS security reqs. or missing Adapers / layered products.	Most scenarios, but requires, quality gates and rollback strategies for critical product issues.
Not Recommended	NOT Recommended as ref environment is not immutable by default and can be accidently changed. Outdated – should not be used anymore.	Not recommended in Kubernetes / non-privileged environments (e.g. OpenShift / Kubernetes build nodes). Deprecated – will be removed in future versions.		Air-gapped environments and special Base OS security reqs.
Patches	Can be applied as required. Including Engineering / Hot fixes.	By default, latest Fixes are applied. Fixes can also be selected. No Engineering / Hot Fix capability	Can be applied as required. Including Engineering / Hot fixes.	Can be provided through Dockerfiles by SAG Support if required.

Classification Internal

webMethods Images on IBM webMethods Container Registry

	Availability
Integration Server / MSR	Yes, see build alternatives for recommendations
My webMethods Server	No
Universal Messaging	Yes
API Gateway	Yes
Developer Portal	Yes
Terracotta Big Memory	Yes
Active Transfer	Yes

Classification - Internal

Custom Image Building Best Practices

- Multi-stage builds: a builder stage compiles and validates, the final stage copies only the result
- Group packages into layers by change frequency, not alphabetically
- Fail the BUILD, not the container: compile Java services and check manifests in the builder stage
- Keep a shared "platform" image (drivers, adapters, framework packages) that business images build FROM
- Never bake application.properties, licences or passwords into an image
- Tag with semantic version AND commit SHA; label with org.opencontainers.image.* metadata

Classification - Internal

Multi-Stage Containerfile (Capstone 5)

block5/capstone/msr/Containerfile

```
ARG MSR_IMAGE=ibmwebmethods.azurecr.io/webmethods-microservicesruntime:12.1.0.3
FROM ${MSR_IMAGE} AS builder          # JDK 21 incl. javac is in the image
USER root
COPY packages/SwisscomLab /build/SwisscomLab
RUN javac --release 21 -cp "$IS_CLASSPATH" -d /build/SwisscomLab/code/classes \
      $(find /build/SwisscomLab/code/source -name '*.java') \
 && test -f /build/SwisscomLab/manifest.v3

FROM ${MSR_IMAGE}
ADD  --chown=1724:1724 ${MAVEN}/…/postgresql-42.7.4.jar $IS/lib/jars/custom/
COPY --chown=1724:1724 block5/capstone/msr/jdbc/PostgreSQL_JDBC_Driver.xml $IS/config/jdbc/driver/
COPY --from=builder --chown=1724:1724 /build/SwisscomLab $IS/packages/SwisscomLab
ARG APP_VERSION
ENV APP_VERSION=${APP_VERSION}
USER 1724
```

$IS = /opt/webmethods/IntegrationServer (abbreviated here). The builder stage never ships: only its compiled, validated output is copied (COPY --from).

Classification - Internal

Layer Optimization Checklist

Layer	Contents	Change frequency	Rebuild cost
1 · IBM base	UBI 9 + JDK 21 + MSR 12.1	Fix levels only	High: avoid touching
2 · Platform	JDBC drivers, driver aliases, adapters (wpm)	Monthly	Medium
3 · Framework packages	Shared utilities, common services	Monthly	Medium
4 · Business packages	Swisscom packages per release unit	Daily / weekly	Low: expected
Runtime (not a layer)	application.properties, secrets, licence	Per deployment	None: mounted

Order the Containerfile like this table, top to bottom: cache hits follow directly.

Classification - Internal

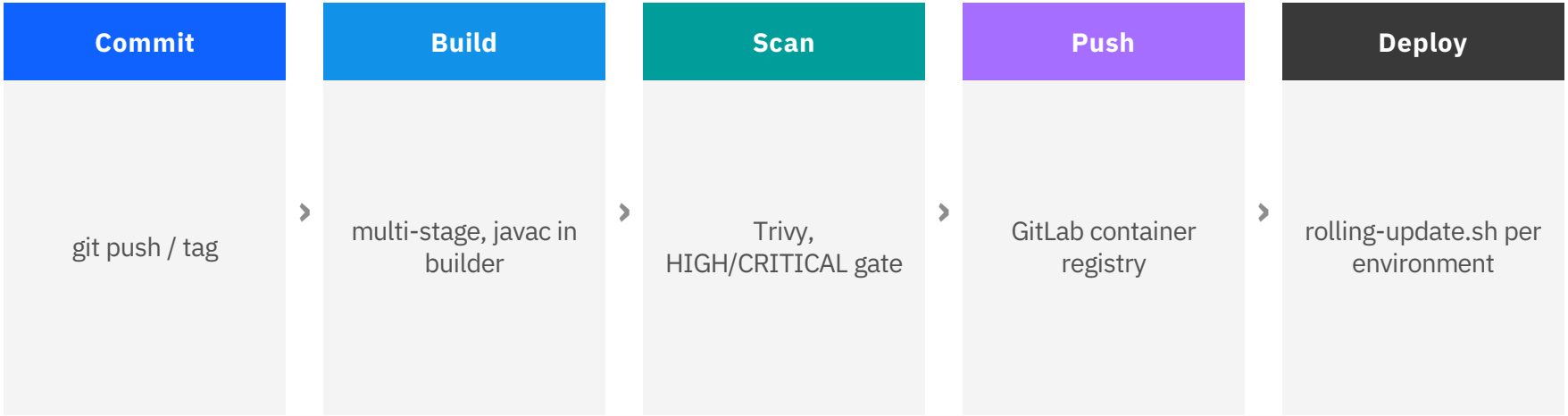
🔧 Hands-On: Exercises 5.1–5.2

5.1	Build block5/capstone/msr/Containerfile; break health.java and watch the builder stage stop the build
5.2	Build 1.1.0: where does the cache stop? Verify label + ENV; move the ARG on a copy: stale version?
5.2	Push 1.0.0 and 1.1.0 to the local registry for the afternoon's rolling update
📁 Files	block5/capstone/msr/Containerfile, packages/SwisscomLab; registry from Exercise 3.5

Participant Workbook, Block 5 · ~30 min · 🏁 both tags carry the right version inside the image

Classification - Internal

GitLab CI/CD Pipeline for MSR Images



Each stage gates the next: a compile error or a fixable critical CVE never reaches the registry.

Base image pulls need the IBM registry token as a masked CI variable.

Classification - Internal

.gitlab-ci.yml: Build, Scan, Push

block5/solution/ci/.gitlab-ci.yml (excerpt)

```
stages: [build, scan, push, deploy]
default: { image: quay.io/podman/stable:v5 }   # no dind service
build-image:
  script:
    - echo "$IBM_REGISTRY_TOKEN" | podman login ibmwebmethods.azurecr.io
        -u "$IBM_REGISTRY_USER" --password-stdin
    - podman build -f block5/solution/msr/Containerfile
        --build-arg APP_VERSION=$CI_COMMIT_SHORT_SHA -t $IMAGE:$TAG .
    - podman save --format docker-archive $IMAGE:$TAG -o msr-image.tar
scan-image:
  image: { name: docker.io/aquasec/trivy:0.56.2, entrypoint: [""] }
  script:
    - trivy image --input msr-image.tar --exit-code 1
        --ignore-unfixed --severity HIGH,CRITICAL
deploy-dev:
  when: manual
  script: ["PROD=1 ./scripts/rolling-update.sh $TAG"]
```

This slots next to Swisscom's existing GitLab pipelines: you add stages, you don't replace the platform.

Classification - Internal

Package & Image Gotchas and Best Practices

Gotchas & pitfalls

- ✗ Implicit package load-order dependencies succeed in one build and fail in the next
- ✗ "Just one more package" turns layer 4 into an undifferentiated monolith
- ✗ Skipping the scan stage is how a known-CVE base image reaches production
- ✗ Tokens passed as build ARGs end up in the image history

Best practices

- ✓ Declare package dependencies in manifest.v3 and test them in CI
- ✓ Split business packages into release units / images
- ✓ Treat the scan as a hard gate, like a failing unit test
- ✓ Use build secrets (--mount=type=secret) for wpm and Git tokens

Classification - Internal

🔧 Hands-On: Exercise 5.3

5.3	The CI gate locally: podman save, Trivy --input, --exit-code 1 --ignore-unfixed HIGH,CRITICAL
💬	The gate failed on IBM's 12.1 base image alone. Swisscom's policy: pin fix tags? .trivyignore with expiry?
📁 **Files**	block5/solution/ci/.gitlab-ci.yml (scan-image job)

Participant Workbook, Block 5 · ~15 min · 🏁 you can say which findings are yours and which are the base image's

Classification - Internal



Swisscom-Specific Architecture

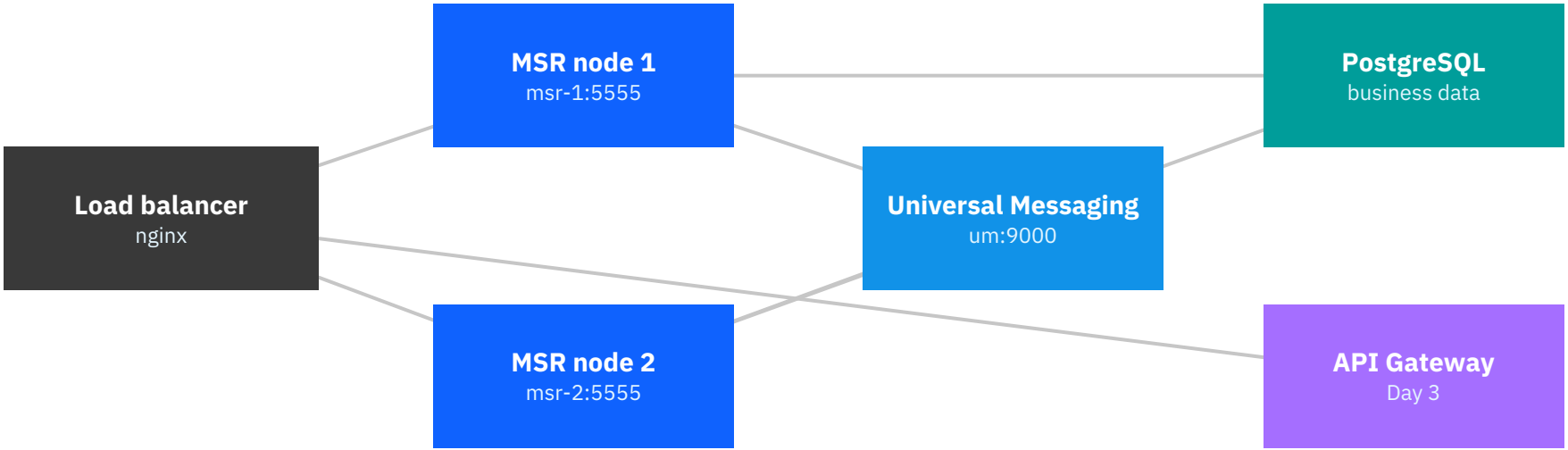
2 instances × 4 environments · Clustering in Compose · Load balancing

Discussion: The 12 Cluster Architecture

- Swisscom runs 12 Integration Server clusters across the platform
- Split by workload/domain and with two IS instances each for redundancy per environment (24 total IS instances)
- Each pair shares a UM cluster and an environment-scoped database
- Containerization question: which instances map 1:1 to a container, which consolidate, which split into smaller MSR images?
- Compose is a single-host tool; true multi-host clustering is where Kubernetes enters (Day 3 outlook)

Classification - Internal

Clustered Setup in Compose (per environment)



Two MSR containers behind nginx, sharing one UM and one database: the shape of one Swisscom environment on one host.

Classification - Internal

Load-Balancing Strategies

Options at this layer

- nginx or HAProxy container in front of N MSR containers
- Round-robin for stateless services (measured in Capstone 5: pairs per nginx worker)
- proxy_next_upstream retries the other node on errors
- Sticky sessions only where a flow depends on IS session state

Where Compose stops helping

- No built-in load balancer: you bring nginx/HAProxy
- Scale (--scale) only on one host; no cross-host scheduling
- No native rolling update or failover: Day 3 does it by hand
- This ceiling is why the Kubernetes engagement is scoped as a follow-on

Classification - Internal

Clustering Gotchas & Best Practices

Gotchas & pitfalls

- ✗ Two nodes with the same JMS clientID compete for one connection
- ✗ nginx resolves service names at start/reload: a recreated node has a new IP
- ✗ Sticky sessions hide state-sharing problems until a node is replaced
- ✗ Scaling on one host gives no real fault tolerance

Best practices

- ✓ One JMS_CLIENT_ID per node, set via environment
- ✓ Reload nginx after every node recreation
- ✓ Stateless services; state in UM/DB
- ✓ Document the Compose topology as the blueprint for Kubernetes

Classification - Internal

webMethods Components in Containers

API Gateway · Authorization · TN/AT · Universal Messaging · JDBC Adapter

API Gateway 11.1 Container: Facts from Capstone 5

Item	Value
Image	ibmwebmethods.azurecr.io/apigateway-minimal:11.1.0.14 (≈ 2 GB, UID 1724, IS-based, /opt/softwareag; no 12.1 image yet)
Data store	External OpenSearch 2.19 (the minimal image has no internal store): apigw_elasticsearch_hosts=apigw-store:9200
Ports	9072 UI, 5555 runtime HTTP, 5543 HTTPS
Admin password	SAG_IS_ADMIN_PASSWORD (default manage is then rejected)
Health	/rest/apigateway/health requires authentication: anonymous calls get 401
Start-up	~110–120 s until healthy: use start_period: 120s
API import	REST: POST /rest/apigateway/apis (OpenAPI file) + PUT .../{id}/activate

Other apigw_* variables configure clustering, the analytics store, SMTP and the control-plane agent.

Classification - Internal

Authorization Server & Custom Packages for API Filtering

- OAuth2 / JWT / API-key enforcement uses API Gateway's built-in policies and its authorization server
- Custom filtering logic ships as IS packages inside the Gateway image: build FROM apigateway-minimal and COPY them
- Client IDs, scopes, token endpoints and keys are environment-specific: externalize them, never bake them in
- Import APIs, policies and applications through the REST API from Git (as register-api.sh does in Capstone 5)
- Test a custom filter in isolation before putting it in the policy chain: a bad filter rejects ALL traffic

Gotcha: A broken filter fails closed: it looks like an outage, not a policy bug. Always keep a policy rollback path.

Classification - Internal

Trading Networks & Active Transfer

- AT 12.1 has public images: webmethods-activetransfer (MSR + WmMFT, ≈ 3 GB) and webmethods-activetransfer-dcc (DB schema); TN has none, so build it FROM the MSR base with wpm and push it to the Swisscom registry
- TN and AT are more stateful than plain IS: partner profiles, document archives, transfer sessions
- TN needs its database schemas (Database Component Configurator); AT needs durable transfer directories on volumes
- A mid-transfer restart can leave a partner file half-delivered: generous stop_grace_period, resumable transfers
- Run TN/AT as dedicated singleton services per environment, not replicated like stateless MSR nodes

Classification - Internal

Universal Messaging in a Container: Revisited

- The data directory (.../umserver/data) is the state that must survive: always a volume
- HA: UM clusters (2–3 realms), each node its own container, volume and REALM_NAME
  - ...but containerized UM is not recommended to run in a cluster – just restart a failed node
- Size with INIT_JAVA_MEM_SIZE / MAX_JAVA_MEM_SIZE inside the container memory limit
- Health: /health/ on 9000; administration with runUMTool.sh or Enterprise Manager on nsp://host:9000
- Watch the volume size: unconsumed queues grow the store
- Queue names without dots (LabOrders, not lab.orders)

Classification - Internal

JDBC Adapter Configuration in Containers

- WmJDBCAdapter is not in the MSR base image: install it at build time with wpm (licensed registry)
- JDBC driver jars go to IntegrationServer/lib/jars/custom, the same place as in Capstone 4
- Connection aliases point at service names / DNS, like the core JDBC pools
- Pool size per container × number of replicas = connections the database must accept
- Credentials via $env{...} from secrets; never in the connection alias baked into the image

Gotcha: With 2 nodes and maxConns=20 per pool, plan 40 database connections per environment, per pool.

Classification - Internal

🏆 Capstone 5: Two MSR Nodes, nginx, API Gateway

Objectives

70 min

- Start the Capstone 4 shape from block5/capstone/ (multi-stage image from 5.1)
- msr-2 from the *msr anchor with its own JMS client ID; no published MSR ports
- nginx load balancer WITHOUT depends_on on the nodes (getent wait loop)
- apigw-store (OpenSearch 2.19) on its own network; API Gateway on both
- Authenticated Gateway healthcheck, start_period 120 s
- register-api.sh; curl-loop through the Gateway hits both nodes

Prerequisites

- Podman machine ≥ 10 GB RAM
- Capstone 4 stack stopped (down -v)
- block5/capstone/.env from .env.example
- Reference: block5/solution/

Deliverable
All services healthy; curl-loop through :5556 shows msr-1 and msr-2; one paragraph mapping this stack to one Swisscom environment. Leave it running for Block 6.

Classification - Internal

💬 Q&A and Discussion: Capstone 5

1. *Which of the 12 clusters / 24 current instances are natural candidates to consolidate, or to split into smaller MSR images?*

2. *Where does session state or stickiness exist today, and can it be removed before migration?*

3. *Which dependencies (DB, UM, partner endpoints) must be healthy before an MSR may take traffic?*

Classification - Internal

Block 5 Recap

- ✓ 400 packages: build-time for frameworks/adapters, decide deliberately for business packages
- ✓ Multi-stage MSR builds compile and validate packages before an image exists
- ✓ Verify what you built: label + ENV checks in CI; scan gates need a base-image policy
- ✓ One Swisscom environment in Compose: 2 × MSR, nginx, shared UM + DB, API Gateway
- ✓ No depends_on from the LB to the nodes: podman-compose would restart it on updates

Coming up next: Block 6: secrets and a production overlay, a zero-downtime rolling update, Graylog, and the road from Windows VMs to Linux containers.

Classification - Internal